TypeScript MCQs with Answers & Reasons

Q1. What is the meaning of `(): void` ?

A) Returns number

B) Takes no arguments and returns nothing

C) Takes arguments

D) Returns undefined only

Answer: B

Reason: `()` = no parameters, `void` = no return value.

Q2. Return type of function returning function?

A) void

B) function

C) () => type

D) =>

Answer: C

Reason: Function types are written as `() => returnType` .

Q3. Type of `[1, 2, '3']` ?

A) number[]

B) string[]

C) (number | string)[]

D) tuple

Answer: C

Reason: Mixed values → union type array.

Q4. Invalid tuple?

A) [1, "2", 3]

B) [1, 2, "3"]

C) ["A", 1]

D) [1, 2]

Answer: B

Reason: Order/type mismatch in tuple.

Q5. `[string, ...number[]]` means?

A) Only string

B) Only numbers

C) First string, rest numbers

D) First number

Answer: C

Reason: Tuple + rest → fixed first, flexible rest.

Q6. Invalid syntax?

A) [string, ...number[]]

B) [number, ...string[]]

C) [string, ...number]

D) [string, number]

Answer: C

Reason: Rest must be an array type (`number[]`).

Q7. Interface feature?

A) No extension

B) No merging

C) Supports merging

D) Only for functions

Answer: C

Reason: Interfaces support declaration merging.

Q8. Default enum start value?

A) 1

B) 0

C) -1

D) undefined

Answer: B

Reason: Enums start from 0 by default.

Q9. Access enum?

A) Direction(Up)

B) Direction.Up

C) Up.Direction

D) Direction->Up

Answer: B

Reason: Dot notation is used to access enum members.

Q10. Valid?

let name = [string, ...string[]];

A) Yes

B) No

C) Partial

D) Only TSX

Answer: B

Reason: `string` is a type, not a value.

Q11. `[string, ...string[]]` means?

A) One string
B) At least one string
C) Two strings
D) Empty only
Answer: B
Reason: First required + rest optional strings.

Q12. Types used where?

A) After =
B) After :
C) Inside {} only
D) Nowhere
Answer: B
Reason: `:` is used for type annotation.

Q13. `let x: number = "10"` ?

A) Valid
B) Invalid
C) Runtime only
D) Depends
Answer: B
Reason: Type mismatch (string ≠ number).

Q14. Type of `["A", "B"]` ?

A) string[]
B) number[]
C) tuple

D) any

Answer: A

Reason: Homogeneous array → string[].

Q15. Tuple means?

A) Dynamic array

B) Fixed structure

C) Only numbers

D) Only strings

Answer: B

Reason: Tuples have fixed types & order.

Q16. Which is tuple?

A) string[]

B) [string, number]

C) number[]

D) any[]

Answer: B

Reason: Tuple uses square brackets with fixed types.

Q17. Rest element used with?

A) number

B) string

C) array type

D) object

Answer: C

Reason: Rest (`...`) must be applied to arrays.

Q18. ... means?

A) spread/rest

B) loop

C) condition

D) type

Answer: A

Reason: Used for spread/rest operations.

Q19. Enum type?

A) function

B) constant set

C) variable

D) class

Answer: B

Reason: Enum defines named constants.

Q20. String enum advantage?

A) Faster

B) Readable

C) Smaller

D) None

Answer: B

Reason: String values are human-readable.

Q21. Reverse mapping works in?

A) string enum

B) numeric enum

C) both

D) none

Answer: B

Reason: Only numeric enums support reverse mapping.

Q22. `type A = string | number` is?

A) tuple

B) union

C) enum

D) object

Answer: B

Reason: `|` creates a union type.

Q23. Interface used for?

A) values

B) types

C) loops

D) enums

Answer: B

Reason: Interfaces define type structure.

Q24. Type alias used for?

A) only objects

B) only arrays

C) flexible types

D) none

Answer: C

Reason: Types support unions, tuples, etc.

Q25. Which is valid?

A) let x: string = 10

B) let x: number = 10

C) let x: boolean = "true"

D) let x: number = "10"

Answer: B

Reason: Type matches value.

Q26. `void` means?

A) number

B) string

C) no return

D) undefined always

Answer: C

Reason: Function returns nothing.

Q27. Function type?

A) string

B) () => number

C) {}

D) []

Answer: B

Reason: Arrow syntax defines function type.

Q28. `[number, number]` means?

A) any numbers

B) exactly 2 numbers

C) strings

D) none

Answer: B

Reason: Tuple defines fixed length.

Q29. `[...number[]]` means?

A) tuple

B) array

C) object

D) enum

Answer: B

Reason: Equivalent to number[].

Q30. Which invalid?

A) number[] = [1,2]

B) string[] = ["a"]

C) number[] = ["a"]

D) boolean[] = [true]

Answer: C

Reason: Type mismatch.

Q31. `any` means?

A) strict

B) no type checking

C) number

D) string

Answer: B

Reason: Disables type safety.

Q32. `unknown` safer than?

A) any
B) string
C) number
D) void
Answer: A
Reason: Requires type checking before use.

Q33. `never` means?

A) always returns
B) never returns
C) string
D) null
Answer: B
Reason: Used for unreachable code.

Q34. Enum keyword?

A) class
B) type
C) enum
D) interface
Answer: C
Reason: `enum` defines enumerations.

Q35. Optional property?

A) name!
B) name?
C) name*

D) name#

Answer: B

Reason: `?` makes property optional.

Q36. Readonly property?

A) readonly name

B) fixed name

C) const name

D) static name

Answer: A

Reason: `readonly` prevents modification.

Q37. Function param optional?

A) x!

B) x?

C) x*

D) x#

Answer: B

Reason: `?` makes parameter optional.

Q38. Arrow function syntax?

A) function =>

B) () =>

C) ->

D) =>()

Answer: B

Reason: Standard arrow function syntax.

Q39. Which is valid?

A) let x: []
B) let x: number[]
C) let x: number
D) all
Answer: D
Reason: All are valid type declarations.

Q40. Type inference means?

A) manual typing
B) automatic typing
C) no typing
D) runtime typing
Answer: B
Reason: TS infers type automatically.

Q41. `[string, number?]` means?

A) 2 required
B) second optional
C) first optional
D) none
Answer: B
Reason: `?` makes second element optional.

Q42. Function returning number?

A) () => void
B) () => number
C) number => ()

D) void => number

Answer: B

Reason: Return type is number.

Q43. Object type?

A) {}

B) []

C) ()

D) enum

Answer: A

Reason: `{}` defines object shape.

Q44. Which correct?

A) let x = string

B) let x: string

C) let x = 10

D) both B & C

Answer: D

Reason: B is declaration, C is inference.

Q45. `const enum` ?

A) runtime

B) compile optimized

C) slow

D) invalid

Answer: B

Reason: Removed during compilation.

Q46. Spread operator?

A) ...
B) :::
C) ***
D) ///
Answer: A
Reason: `...` is spread/rest operator.

Q47. Array of numbers?

A) number
B) number[]
C) []number
D) array
Answer: B
Reason: Standard array syntax.

Q48. Tuple vs array?

A) same
B) tuple fixed
C) array fixed
D) none
Answer: B
Reason: Tuple = fixed, array = flexible.

Q49. Interface keyword?

A) type
B) interface
C) enum

D) class

Answer: B

Reason: Used to define structure.

Q50. Best rule?

A) = for types

B) : for values

C) : for types

D) none

Answer: C

Reason: `:` is used for type annotation.